

CREATE A NODEJS AND DEPLOY TO HEROKU

Adapted from:

[How to deploy a NodeJS app to Heroku from Github \(without installing Heroku on your machine\)](https://www.freecodecamp.org/how-to-deploy-a-nodejs-app-to-heroku-from-github-without-installing-heroku-on-your-machine)
([freecodecamp.org](https://www.freecodecamp.org))

STEP 1: CREATE PACKAGE.JSON FILE

Create a new folder and open that folder in terminal

Open terminal and type: *npm init*

Keep hitting Enter to accept all default values

An example package.json could be like this

```
{
  "name": "coolnodeapp",
  "version": "1.0.0",
  "description": "node app ",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "repository": {
    "type": "git",
    "url": ""
  },
  "author": "",
  "license": "ISC",
  "bugs": {
    "url": ""
  },
  "homepage": ""
}
```

One very important change to notice is this line:

```
"start": "node app.js"
```

STEP 2. ADD A FILE, APP.JS, AND PASTE THE BELOW CODE

Notice: app.js and package.json in the same folder

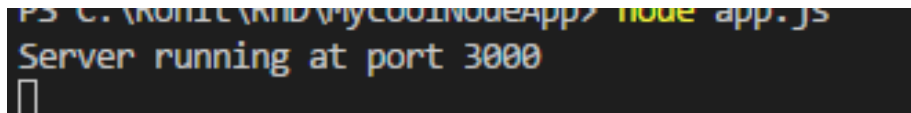
```
const http = require('http');
const port = process.env.PORT || 3000

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/html');
  res.end('<h1>Hello World</h1>');
});

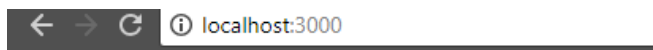
server.listen(port, () => {
  console.log(`Server running at port `+port);
});
```

You can test the app on local by typing at the terminal

```
node app.js
```

A terminal window with a dark background. The prompt is 'PS C:\Kohit\KMD\myCoolNodeApp>'. The command 'node app.js' has been entered. The output is 'Server running at port 3000'. A cursor is visible on the line below the output.

Open browser and run: <http://localhost:3000/>

A browser address bar with a dark background. It contains navigation icons (back, forward, refresh) and the text 'localhost:3000'.

Hello World

STEP 3: PUSH TO GITHUB

Go to Github and create a new repository like this

Create a new repository

A repository contains all the files for your project, including the revision history.

Owner

 rramname ▾

Repository name

/ MyCoolNodeApp 

Great repository names are short and memorable. Need inspiration? How about **vigilant-octo-eureka**.

Description (optional)



Public

Anyone can see this repository. You choose who can commit.



Private

You choose who can see and commit to this repository.



Initialize this repository with a README

This will let you immediately clone the repository to your computer. Skip this step if you're importing an existing repository.

Add .gitignore: **None** ▾

Add a license: **None** ▾



Create repository

GitHub will create a repository and give you some commands that you can run locally so that you can clone your local folder with your GitHub repository.

Navigation: <> Code | Issues 0 | Pull requests 0 | Projects 0 | Wiki | Insights | Settings

Quick setup — if you've done this kind of thing before

Set up in Desktop or **HTTPS** SSH

We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# MyCoolNodeApp" >> README.md
git init
git add README.md
git commit -m "first commit"
git remote add origin https://github.com/rname/MyCoolNodeApp.git
git push -u origin master
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/rname/MyCoolNodeApp.git
git push -u origin master
```

...or import code from another repository

You can initialize this repository with code from a Subversion, Mercurial, or TFS project.

[Import code](#)

ProTip! Use the URL for this page when adding GitHub as a remote.

In the same path with **app.js** and type in the terminal

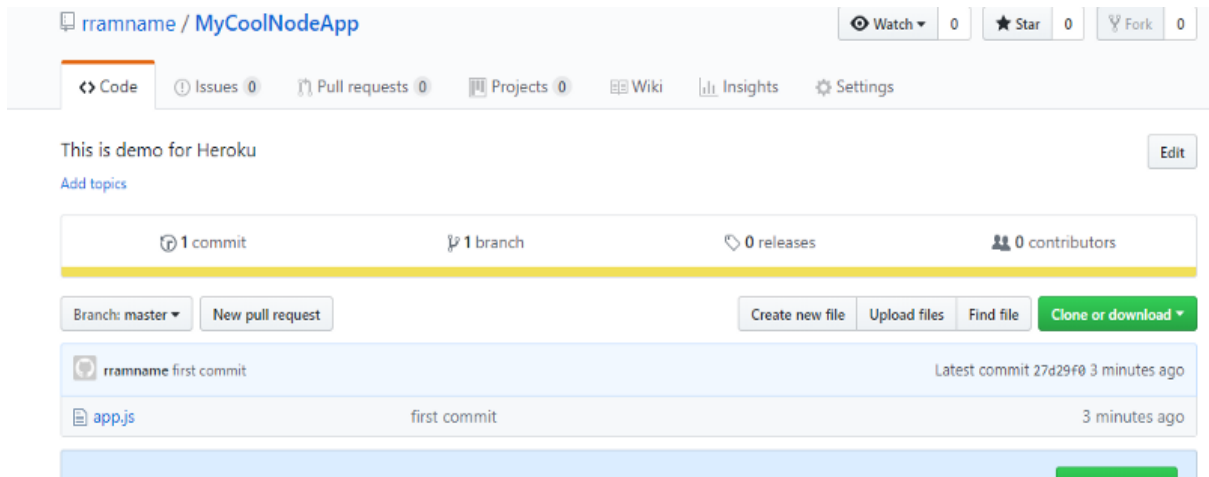
(Remember to replace your URL in remote address. Don't use MyCoolNodeApp URI)

```
git init
git add .
git remote add origin https://github.com/rname/MyCoolNodeApp.git
git push --set-upstream origin master
```

You should see messages like below at the command prompt

```
Counting objects: 3, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 414 bytes | 414.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0)
To https://github.com/rname/MyCoolNodeApp.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
```

Now if you open GitHub and refresh the repository, you should be able to see the code.



STEP 4 DEPLOY TO HEROKU

If you don't have an account with Heroku, you can open a free one by filling out this [simple form](#).
(And here, you don't need to provided Credit Card information :))

Sign up for free and experience Heroku today

 **Free account**

Create apps, connect databases and add-on services, and collaborate on your apps, for free.

 **Your app platform**

A platform for apps, with app management & instant scaling, for development and production.

 **Deploy now**

Go from code to running app in minutes. Deploy, scale, and deliver your app to the world.

First name *

Last name *

Email address *

Company name

Role *

Country *

Primary development language *

☐ I'm not a robot 

[Privacy](#) [Terms](#)

CREATE FREE ACCOUNT

Once you have your account ready, login with your credentials.

Click New on the top right corner and select "Create new app".

Give your app a name (This will be included in the public URL for your application) and click Create app.

This step will take you to the dashboard of your app. Open Deploy tab and scroll to the "Deployment method" section.

Select GitHub as the method.

Deployment method

 Heroku Git
Use Heroku CLI

 GitHub
Connect to GitHub

 Container Registry
Use Heroku CLI

Connect to GitHub

Connect this app to GitHub to enable code diffs and deploys.

Search for a repository to connect to

Missing a GitHub organization? [Ensure Heroku Dashboard has team access](#)

 ramname/MyCoolNodeApp

If it's able to find and connect to the GitHub repository, the Deployment section will show up where you can select if you want Automatic Deployment (as soon as the changes are pushed to GitHub, Heroku will pick them up and deploy) or Manual Deployment.

The screenshot shows the Heroku Deployment section. Under the 'Automatic deploys' heading, there is a description: 'Enables a chosen branch to be automatically deployed to this app.' To the right, under 'Enable automatic deploys from GitHub', it says: 'Every push to the branch you specify here will deploy a new version of this app. Deploys happen automatically: be sure that this branch is always in a deployable state and any tests have passed before you push. Learn more.' Below this is a dropdown menu with 'master' selected. There is a checkbox for 'Wait for CI to pass before deploy' with a note: 'Only enable this option if you have a Continuous Integration service configured on your repo.' At the bottom of this section is a button labeled 'Enable Automatic Deploys'. Below this is the 'Manual deploy' section with the description: 'Deploy the current state of a branch to this app.' To the right, under 'Deploy a GitHub branch', it says: 'This will deploy the current state of the branch you specify below. Learn more.' Below this is a dropdown menu with 'master' selected and a button labeled 'Deploy Branch'.

Click Enable Automatic Deploys (because it's less overhead for demo apps :)). You can also select the GitHub branch if you need to, but for this demo we will deploy from the master branch.

Now we need to tell Heroku that our app is a NodeJs app. For that, we will need the NodeJs build back.

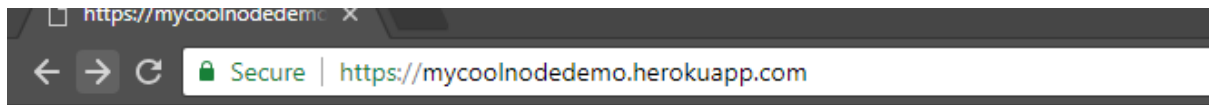
Open the Settings tab and locate Buildpacks and click **“Add buildpack”**.

The screenshot shows the 'Add Buildpack' dialog box. At the top, it says 'Add Buildpack' with a close button. Below is a text input field labeled 'Enter Buildpack URL' with a placeholder: 'e.g. heroku/nodejs or https://github.com/heroku/heroku-buildpack-ruby'. Below this is the text 'Or select from our officially supported buildpacks'. There are ten icons representing different buildpacks: nodejs, python, php, ruby, java, go, gradle, scala, and clojure. At the bottom of the dialog is a button labeled 'Save changes'. In the background, there is a button labeled 'Add buildpack' and a text label 'Buildpacks will appear here'.

Select **nodejs** from the options and click Save changes.

Now, go back to the Deploy tab, and click **Deploy Branch** at the bottom.

Open the **settings** tab and scroll down to the **Domains and certificates** section. Here, you can see the URL of your app that was just deployed. Copy and paste that URL in the browser and... Hooooorah!!



Hello World